

Generate and replication

Verilog gives you two related ideas that sound like loops but run at elaboration,
not on every clock edge

Elaboration, not runtime

- Replication is expression math: brace-N-brace expr widens a bus by repeating expr
- Bit cell open-bracket i close-bracket dot u, and so on
- Generate if tests a parameter or localparam at elaboration and discards the other branch
- That is different from a for loop inside always at-star, which runs during simulation
- Generate conditions must be constants known before simulation starts

Browser lab

Digital Design and Verification Platform

HomeTools

Home / Tools / Generate / replication

INTERACTIVE TOOL

Generate / replication explorer

Expand `{N{expr}}`, unroll `generate for` for instance arrays, and branch with `generate if` — all at **elaboration**, not runtime.

Starter example: replication `{4{1'b1}}` → `{1'b1, 1'b1, 1'b1, 1'b1}` (width 4). Then try `generate for` / `if`.

Load starter example

Challenges (1/22)

Quiz: syntax. Replication uses braces like? Answer: `{N{e}}`

Your answer

Show hintCheckNextIdle

quiz-repquiz-genvarquiz-elabquiz-runtimestarterexpand-repwidth-4rep-2bpreset-forfor-name

if-wideif-narrowexplainmode-forquiz-blockn-changequiz-concatfor-n2quiz-paramdiscard

preset-rep-btnfull-path

Core ideas

Replication

`{N{e}}` copies expression `e` `N` times in a concat.

generate for

`genvar` loop elaborates an instance per index.

generate if

Parameter test picks one structural branch — not runtime.

Explorer

Result

Generate replication lab starter

learn_verilog — sv generate

Real Verilog practice

```
wsl - module12-sv-generate/examples/navigation

# Track A - real Unix
# real Verilog session (Track A)

$ cat examples/generate_demo/generate_demo.v
// generate_demo.v - replication + generate for
module leaf_cell(
    input d,
    output q
);
    assign q = d;
endmodule

module generate_demo #(
    parameter N = 4
)(
    input [N-1:0] d,
    output [N-1:0] q
);
    wire [N-1:0] ones;
    assign ones = {N{1'b1}};

    genvar i;
    generate
        for (i = 0; i < N; i = i + 1) begin : bit_cell
            leaf_cell u (.d(d[i]), .q(q[i]));
        end
    endgenerate
endmodule

$ iverilog -t null examples/generate_demo/generate_demo.v
(syntax check passed)
```

Real Verilog - replication and generate for

Pitfalls to watch

- Do not confuse replication brace-N-brace with generate for, they solve different problems
- Do not put a runtime signal in a generate if condition, use parameters or localparams
- Do not expect generate for to behave like a procedural for loop inside always
- Do name generate blocks when you need hierarchical paths like bit_cell open-bracket i

Your turn

- Complete the checklist for at least one track, preferably both
- In the browser, expand brace-four-brace one-prime-b-one and state the resulting bit width
- On real Verilog, write a generate for that instantiates N copies of a leaf cell
- When you are ready